

PDF

Lab_4_VibeCoding_Student_handout

this is my repo link , what i am i supposed to do and what r the deliveables about and guide me to help me finish it

For Lab 4, you are tasked with using AI prompting ("vibe coding") to repair a broken Python match-three gem game and implement new features from your assigned repository.

PNG + 1

What You Need to Do

- **Fork and clone**: Fork your assigned repository (`[https://github.com/SETAPESU26/57_match_three_gem](https://github.com/SETAPESU26/57_match_three_gem)`) to your personal GitHub profile and clone it locally.
  PNG + 1
- **Record "before" state**: Run the initial Python game and record a 10-second video showing the broken behavior before making any changes to the code.
  PDF
- **Check the README**: Read the repository's `README.md` to identify the broken code sections and list of required features.
  PDF
- **Prompt to fix and build**: Use an AI assistant to fix bugs and build each feature sequentially, aiming for completion in under 3 to 4 prompt iterations per task.
  PDF
- **Commit individually**: Create a separate Git commit for each individual bug fix and feature implementation.
  PDF
- **Record "after" state**: Record a second 10-second video demonstrating the game working properly with all new features active.
  PDF
- **Push to your repo**: Push all changes to your personal repository, ensuring you do not submit a pull request to `SETAPESU26` .
  PDF

Required Deliverables

All deliverables must be added to your personal lab repository inside a folder named `Lab-4`:

PDF

- **Screen recordings:** The two 10-second video files demonstrating the game before and after modifications.
  PDF
- **Updated source code:** The complete, functioning Python game repository with distinct commits for each task.
  PDF
- **Chat transcript**: Your exported AI interaction history formatted as a PDF or DOC file.
  PDF

Step-by-Step Execution Guide

Start by cloning the repository to your machine, running the main game file, and capturing your initial 10-second video. Once recorded, inspect the repository's `README.md` to see the exact bugs and feature checklist assigned to your repo.

What does the `README.md` in your repository specify as the broken items and feature requirements?

how to fork and clone locally , in terminal

Forking is a GitHub platform feature, so you can fork via the GitHub website (recommended) or entirely via the terminal using the GitHub CLI (`gh`) before cloning locally.

Method 1: Web Fork + Terminal Clone (Standard)

1. **Fork the repository in your browser**
    1. Visit the repository URL: `[https://github.com/SETAPESU26/57_match_three_gem](https://github.com/SETAPESU26/57_match_three_gem)` [cite: 1].
    2. Click the **Fork** button in the top-right corner.
    3. Leave the repository name as is, select your personal account as the owner, and click **Create fork**.

4. *Verification:* Check that the URL in your browser updates to `[https://github.com/](https://github.com/)<YOUR_USERNAME>/57_match_three_gem`.

2. **Clone your fork in your terminal**
    1. Open your terminal and run:

```bash
git clone https://github.com/<YOUR_USERNAME>/57_match_three_gem.git
```

    (Replace `<YOUR_USERNAME>` with your actual GitHub username).

    2. *Verification:* Run `cd 57_match_three_gem && ls` (or `dir` on Windows) to verify that the project files appear in your terminal.
3. **Verify the remote origin**
    1. Check where Git pushes your work:

```bash
git remote -v
```

    2. *Verification:* Both `origin (fetch)` and `origin (push)` must display your personal GitHub URL, not `SETAPESU26`.

Method 2: Entirely in Terminal (Using GitHub CLI)

If you have the GitHub CLI installed and authenticated:

1. **Fork and clone in a single command**

```bash
gh repo fork SETAPESU26/57_match_three_gem --clone
```

    1. Select **Yes** when prompted to clone the repository locally.
    2. *Verification:* Run `cd 57_match_three_gem && git remote -v` to confirm your personal fork is set as `origin`.

i have saved it in a d drive under the file name lab04, i set my terminal to point to the given file loaction

To proceed with the lab from your `D:\lab04` folder, follow these steps to inspect the repository, run the code, and capture your required "before" video:

PDF

① Check Repository Files

Terminal

List the contents of the folder to identify the game file (usually `main.py` or `game.py`) and dependencies:

```
DOS

dir
```

Verification: Confirm you see `README.md`, a Python script (such as `main.py`), and any requirements file listed in the output.

② Inspect the README Requirements

Terminal or Editor

Read the task description and requirements directly in your terminal:

```
DOS

type README.md
```

(Alternatively, open the folder in VS Code with `code .` *or view* `README.md` *in Notepad).*

Verification: You should see the list of broken features to fix and new features to implement.

③ **Install Dependencies & Run the Game**

Record Video

1. If there is a `requirements.txt`, install dependencies first:

```
DOS

pip install -r requirements.txt
```

(If `pygame` *is needed and not listed, install it directly:* `pip install pygame`*).*
2. Start screen recording (using **Win + Alt + R** for Windows Game Bar or your recording tool of choice).
3. Run the Python game:

```
DOS

python main.py
```

(Replace `main.py` *with the actual script name if different).*
4. Let the game run for at least 10 seconds to capture the initial broken behavior, then stop recording and save the file.

Verification: Check that the game window launches (or outputs the expected starting bug/crash) and your 10-second MP4 video is saved.

Once you have recorded the video, paste the contents of `README.md` (or the console error/file list) here so we can start fixing the broken code and building the features task by task.

PDF



what to do?

Before modifying the code, ensure you have captured your initial deliverable:

PDF

1. Record the "Before" Video

Your lab instructions require a 10-second recording of the game before any changes are made. Record your screen while running `python main.py` in PowerShell showing this crash, which documents the initial broken state.

PDF + 1

2. Locate the Bug in `game/board.py`

The traceback points directly to a variable name mismatch on line 119:

Plaintext

```
File "D:\sem5\se_lab\lab04\game\board.py", line 119, in drop_and_refill
  if self.grid[r][c] is None:
NameError: name 'c' is not defined
```

To see the surrounding code in PowerShell, run:

PowerShell

```
Get-Content -Path .\game\board.py | Select-Object -Skip 110 -First 25
```

Or open `game/board.py` in VS Code:

PowerShell

```
code game/board.py
```

3. Identify and Apply the Fix

Look at the loop structure in `drop_and_refill()` around lines 110–125. The loop variable for columns was likely named `col`, `col_idx`, or `x`, but line 119 references `c`:

- If the loop header is `for col in range(...):`, change `c` to `col` in `self.grid[r][c]` (or change the loop variable to `c`).
- Save the file and test again:

PowerShell

```
python main.py
```

4. Commit the Bug Fix

Once the game starts running or moves past this crash, commit this change separately as required by the handout:

PDF

PowerShell

```
git add game/board.py
```

```
git commit -m "Fix NameError in drop_and_refill by defining column variable"
```

Also, run `type README.md` in your terminal and share its contents here so we can tackle the rest of the required feature list in sequence.

PDF

done i have fixed it and have recored the 10 to 15 second before video .PS D:\sem5\se_lab\lab04> type README.md
Match-3 Gem Swap Repair Lab

Here is the roadmap based on your `README.md` and lab handout:

```
Crash Fix (Completed) → Task 1 (Invalid Swap Bug) → Task 2 (Cascade Multiplier) → Ta
```

The lab instructions require a **separate Git commit for each task**.

PDF

Step 1: Commit your initial crash fix

If you haven't committed the `NameError: name 'c' is not defined` fix yet, run this in your terminal:

PowerShell

```
git add game/board.py
git commit -m "Fix NameError c undefined in drop_and_refill"
```

Step 2: Tackle Task 1 — Fix Invalid Swap Move Deduction

Bug summary: Swapping two gems that do not produce a match reverts them to their starting tiles, but still decreases the remaining move count. Moves should only decrement when a valid match is formed.

To fix this, check where the swap validation and `moves` decrement take place. Look inside either `game/board.py` or `game/game_engine.py`.

Run this in your PowerShell terminal to inspect where `moves` is modified:

PowerShell

```
Select-String -Path .\game\*.py -Pattern "moves"
```

Or open the `game/` folder in your editor and share the `swap` / `try_swap` / `handle_click` function here. Once you paste that section of code, we will write the exact fix for Task 1.



here u go

Notice line 154 in `game\board.py`:

PNG

Python

```
self.moves_remaining -= 1
```

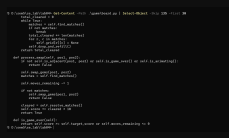
This is where the moves are currently being deducted regardless of whether a valid match occurred.

To view the exact function surrounding line 154, run this command in your PowerShell terminal:

PowerShell

```
Get-Content -Path .\game\board.py | Select-Object -Skip 135 -First 30
```

Paste that snippet here, and we will adjust the condition so `self.moves_remaining -= 1` only executes when a swap produces a valid match.



This code snippet of the following

In `game/board.py`, `self.moves_remaining -= 1` is being executed before checking whether any matches exist. When a swap fails, the gems revert, but the move count is already docked.

PNG + 1

Task 1 Solution: Move the Deduction After the Match Check

Open `game/board.py` and replace `process_swap` with the following:

Python

```
    def process_swap(self, pos1, pos2):
        if not self.is_adjacent(pos1, pos2) or self.is_game_over() or self.is_animati
            return False

        self.swap_gems(pos1, pos2)
        matches = self.find_matches()
```

```
        if not matches:
            self.swap_gems(pos1, pos2)
            return False

        # Only deduct a move when a valid match is made
        self.moves_remaining -= 1

        cleared = self.resolve_matches()
        self.score += cleared * 10
        return True
```

Verify and Commit Task 1

1. **Test the fix:** Run `python main.py`. Swap two adjacent gems that **do not** create a match. Verify that the gems revert and your `MOVES LEFT` counter stays the same. Then make a valid match and confirm the counter decrements by 1.
2. **Commit Task 1:** In PowerShell, commit this specific task:
   PDF

PowerShell

```
git add game/board.py
git commit -m "Task 1: Fix invalid swap move deduction bug"
```

Next Up: Task 2 (Cascade Combo Multiplier)

Task 2 requires multiplying score rewards sequentially for every consecutive cascade/fall chain reaction within a single turn.

Look at the `resolve_matches` loop directly above `process_swap` in your screenshot:

PNG

Python

```
        total_cleared = 0
        while True:
            matches = self.find_matches()
            if not matches:
                break
```

```
            total_cleared += len(matches)
            for r, c in matches:
                self.grid[r][c] = None
            self.drop_and_refill()
        return total_cleared
```

Notice that `resolve_matches()` currently returns just `total_cleared`, and `process_swap()` calculates `self.score += cleared * 10`. To reward cascade combos, the scoring multiplier must increment inside that `while True:` loop for each cascade wave.

PNG

Ready to implement Task 2?

yes lets head to task 2

Task 2: Implement Cascade Combo Multiplier

In `game/board.py`, `resolve_matches()` clears matches in a loop with `drop_and_refill()`, but currently only counts `total_cleared` with a flat 10 points per gem.

PNG

To award increasing points for consecutive cascades in a turn, track a `combo` multiplier starting at 1 that increments with each cascade wave.

1. Update `resolve_matches` and `process_swap` in `game/board.py`

Replace `resolve_matches` and `process_swap` in `game/board.py` with:

Python

```
    def resolve_matches(self):
        total_points = 0
        combo = 1

        while True:
            matches = self.find_matches()
```

```
            if not matches:
                break

            # Each wave scales with the current cascade multiplier (1x, 2x, 3x, ...)
            wave_cleared = len(matches)
            total_points += wave_cleared * 10 * combo
            combo += 1

            for r, c in matches:
                self.grid[r][c] = None
            self.drop_and_refill()

        return total_points

    def process_swap(self, pos1, pos2):
        if not self.is_adjacent(pos1, pos2) or self.is_game_over() or self.is_animati
            return False

        self.swap_gems(pos1, pos2)
        matches = self.find_matches()

        if not matches:
            self.swap_gems(pos1, pos2)
            return False

        self.moves_remaining -= 1

        # Add points earned including cascade multipliers
        points_earned = self.resolve_matches()
        self.score += points_earned
        return True
```

2. Test the Implementation

Run the game:

PowerShell

```
python main.py
```

- **Verify:** Make a match that triggers a chain reaction (gems falling and immediately matching another 3+ set). The score will increase by 20 points per gem on the second cascade wave, 30 points on the third, etc., instead of the flat 10.

3. Commit Task 2

Once verified, commit this task separately:

PDF

PowerShell

```
git add game/board.py
git commit -m "Task 2: Implement cascade combo multiplier for chain reactions"
```

Ready for **Task 3: Implement 4-in-a-row special line-clear gems**?
To implement this smoothly, show how gems are represented (run `Get-Content -Path .\game\board.py | Select-Object -Skip 65 -First 40` or inspect `find_matches`).

task 3

To implement **Task 3: 4-in-a-row special line-clear gems**, we need to know how gems are stored (e.g., as simple integers/colors or as objects/dicts) and how `find_matches` is currently written.

Run this command in PowerShell to inspect `find_matches` and how new gems are created:

PowerShell

```
Get-Content -Path .\game\board.py | Select-Object -Skip 60 -First 70
```

Paste the terminal output here, and we'll implement the 4-in-a-row detection, special gem creation, and row/column detonation logic.

PS D:\sem5\se_lab\lab04> Get-Content -Path .\game\board.py | Select-Object -Skip 60 -First 70

def is_animating(self):

To implement Task 3 cleanly, we also need to see how a `Gem` is defined and drawn (so we can give the 4-in-a-row gem a glowing/special indicator and handle its detonation).

Run this command in PowerShell to inspect where `Gem` is defined:

PowerShell

```
Select-String -Path .\game\*.py -Pattern "class Gem"
```

Also check the top lines of `game/board.py` and where gems are drawn:

PowerShell

```
Get-Content -Path .\game\board.py | Select-Object -First 35
```

Paste what that returns, and we'll assemble the complete 4-in-a-row creation and detonation logic.

PS D:\sem5\se_lab\lab04> Select-String -Path .\game*.py -Pattern "class Gem"

game\board.py:16:class Gem:

To complete **Task 3**, we need to modify three areas in `game/board.py`:

1. Give `Gem` a special flag (e.g., `self.special = True`) and a visual glow effect when drawn.
2. In match detection, identify 4-in-a-row lines: when cleared, spawn a special line-clear gem at that location instead of leaving it empty.
3. When a special gem is matched, detonate the entire row or column.

To make sure our drawing code matches your game's current graphics, run this quick command in PowerShell to see how gems are rendered:

PowerShell

```
Get-Content -Path .\game\board.py | Select-String -Pattern "def draw" -Context 0, 20
```

Paste that snippet, and we'll provide the exact code updates for Task 3.

when the given cmd ran Get-Content -Path .\game\board.py | Select-String -Pattern "def draw" -Context 0, 20 it gives no output and heads to the next line

The drawing logic is in `game/game_engine.py` instead of `board.py`.

Run this command in PowerShell to locate where gems and the board are drawn:

PowerShell

```
Select-String -Path .\game\*.py, .\*.py -Pattern "draw"
```

Or view the render/draw loop in `game/game_engine.py`:

PowerShell

```
Get-Content -Path .\game\game_engine.py | Select-Object -First 60
```

Paste that snippet here so we can see how gems are rendered and add the glowing special-gem visual alongside the detonation logic.

PS D:\sem5\se_lab\lab04> Select-String -Path .\game*.py, .*.py -Pattern "draw"

game\board.py:193: pygame.draw.rect(surface, (20, 2

To make sure our changes integrate smoothly, let's look at lines 180 to 225 in `game/board.py` (which handles drawing the gems):

PowerShell

```
Get-Content -Path .\game\board.py | Select-Object -Skip 180 -First 45
```

Run that command and paste the output. Once we see how each gem is drawn, we will write the exact changes for `Gem`, `find_matches`, `resolve_matches`, and the glowing line-clear rendering.

PS D:\sem5\se_lab\lab04> Get-Content -Path .\game\board.py | Select-Object -Skip 180 -First 45
        return None

Task 3: Implement 4-in-a-Row Special Line-Clear Gems

To fulfill Task 3, we will modify `game/board.py` to:

1. Allow `Gem` objects to hold a `special` property (`'row'` or `'col'`).
2. Detect 4-in-a-row matches and merge them into a special gem at the match location.
3. Detonate the entire row or column when a special gem is subsequently matched.
4. Render a glowing pulsing border and line-clear stripe on special gems.

1. Update `game/board.py`

Open `game/board.py` and apply the following updates:

A. Add `import math` at the top of `game/board.py`

Python

```python
import random
import math
import pygame
```

B. Update `Gem.__init__` (around line 17)

Add the optional `special=None` parameter:

Python

```python
class Gem:
    def __init__(self, color, target_row, col, special=None):
        self.color = color
        self.target_row = target_row
        self.col = col
        self.current_y = (target_row - 2) * TILE_SIZE
        self.target_y = target_row * TILE_SIZE
        self.fall_speed = 12.0
        self.special = special  # None, 'row', or 'col'

    def update(self):
        if self.current_y < self.target_y:
            self.current_y += self.fall_speed
            if self.current_y > self.target_y:
                self.current_y = self.target_y

    def is_animating(self):
        return self.current_y < self.target_y
```

C. Update `Board.__init__` and `Board.reset`

In `__init__`, add state variables:

Python

```
        self.selected = None
        self.last_swap = None
        self.is_initializing = False
        self.reset()
```

In `reset()`, set `self.is_initializing` around `resolve_matches()` so initial board generation does not spawn special gems:

Python

```
    def reset(self):
        self.score = 0
        self.moves_remaining = self.max_moves
        self.selected = None
        self.last_swap = None
        self.is_initializing = True

        for r in range(GRID_SIZE):
            for c in range(GRID_SIZE):
                self.grid[r][c] = Gem(random.choice(GEM_COLORS), r, c)

        self.resolve_matches()
        self.is_initializing = False
```

D. Update `process_swap`

Track `self.last_swap` so the special gem appears where the player made the swap:

Python

```
    def process_swap(self, pos1, pos2):
        if not self.is_adjacent(pos1, pos2) or self.is_game_over() or self.is_animati
            return False

        self.last_swap = (pos1, pos2)
        self.swap_gems(pos1, pos2)
        matches = self.find_matches()

        if not matches:
            self.swap_gems(pos1, pos2)
```

```
            self.last_swap = None
            return False

        self.moves_remaining -= 1
        points_earned = self.resolve_matches()
        self.score += points_earned
        self.last_swap = None
        return True
```

E. Replace `find_matches` and `resolve_matches`

Replace `find_matches` and `resolve_matches` with the full detection, line-detonation, and special gem creation logic:

Python

```
    def find_matches_and_specials(self):
        matched = set()
        specials_to_create = []

        # 1. Horizontal runs
        for r in range(GRID_SIZE):
            c = 0
            while c < GRID_SIZE:
                gem = self.grid[r][c]
                if not gem:
                    c += 1
                    continue
                color = gem.color
                run = [(r, c)]
                k = c + 1
                while k < GRID_SIZE and self.grid[r][k] and self.grid[r][k].color ==
                    run.append((r, k))
                    k += 1

                if len(run) >= 3:
                    matched.update(run)
                    if len(run) >= 4 and not self.is_initializing:
                        # Pivot at swapped tile if available, else middle tile
                        pivot = run[len(run) // 2]
                        if self.last_swap:
                            for swap_pos in self.last_swap:
                                if swap_pos in run:
                                    pivot = swap_pos
                                    break
```

```
                    specials_to_create.append((pivot[0], pivot[1], color, 'row'))
            c = k

        # 2. Vertical runs
        for c in range(GRID_SIZE):
            r = 0
            while r < GRID_SIZE:
                gem = self.grid[r][c]
                if not gem:
                    r += 1
                    continue
                color = gem.color
                run = [(r, c)]
                k = r + 1
                while k < GRID_SIZE and self.grid[k][c] and self.grid[k][c].color ==
                    run.append((k, c))
                    k += 1

                if len(run) >= 3:
                    matched.update(run)
                    if len(run) >= 4 and not self.is_initializing:
                        pivot = run[len(run) // 2]
                        if self.last_swap:
                            for swap_pos in self.last_swap:
                                if swap_pos in run:
                                    pivot = swap_pos
                                    break
                        specials_to_create.append((pivot[0], pivot[1], color, 'col'))
                r = k

        # 3. Special gem detonations (BFS trigger for chain detonations)
        detonated = set(matched)
        queue = list(matched)
        while queue:
            qr, qc = queue.pop(0)
            q_gem = self.grid[qr][qc]
            if q_gem and q_gem.special:
                if q_gem.special == 'row':
                    for ci in range(GRID_SIZE):
                        if (qr, ci) not in detonated and self.grid[qr][ci]:
                            detonated.add((qr, ci))
                            queue.append((qr, ci))
                elif q_gem.special == 'col':
                    for ri in range(GRID_SIZE):
                        if (ri, qc) not in detonated and self.grid[ri][qc]:
                            detonated.add((ri, qc))
                            queue.append((ri, qc))

        return detonated, specials_to_create
```

```python
    def find_matches(self):
        matches, _ = self.find_matches_and_specials()
        return matches

    def resolve_matches(self):
        total_points = 0
        combo = 1

        while True:
            matches, specials_to_create = self.find_matches_and_specials()
            if not matches:
                break

            wave_cleared = len(matches)
            total_points += wave_cleared * 10 * combo
            combo += 1

            for r, c in matches:
                self.grid[r][c] = None

            # Spawn newly created special gems
            for r, c, color, spec_type in specials_to_create:
                new_gem = Gem(color, r, c, special=spec_type)
                new_gem.current_y = r * TILE_SIZE
                self.grid[r][c] = new_gem

            self.drop_and_refill()

        return total_points
```

F. Update `render` in `game/board.py`

Inside the gem drawing loop in `def render(self, surface):`, add the glowing border and line-clear stripe indicator:

Python

```python
                    pygame.draw.rect(surface, gem.color, tile_rect, border_radius=10)
                    pygame.draw.rect(
                        surface, (255, 255, 255), tile_rect, width=1, border_radius=1
                    )

                    # Enhanced glowing gem rendering
                    if gem.special:
```

```
                pulse = int(180 + 75 * math.sin(pygame.time.get_ticks() * 0.0
                glow_color = (255, 255, max(0, min(255, pulse)))
                pygame.draw.rect(surface, glow_color, tile_rect, width=3, bor

                cx, cy = tile_rect.centerx, tile_rect.centery
                if gem.special == 'row':
                    pygame.draw.line(surface, (255, 255, 255), (tile_rect.lef
                elif gem.special == 'col':
                    pygame.draw.line(surface, (255, 255, 255), (cx, tile_rect
                pygame.draw.circle(surface, (255, 255, 255), (cx, cy), 5)
```

2. Test the Implementation

Run the game:

PowerShell

```
python main.py
```

- Match 4 gems in a line: verify that 3 clear and 1 transforms into a glowing gem with a stripe indicator.
- Match the special gem with 2 other gems of the same color: verify that the entire row (or column) detonates and clears.

3. Commit Task 3

Once tested, commit this task separately:

PDF

PowerShell

```
git add game/board.py
git commit -m "Task 3: Implement 4-in-a-row special line-clear gems"
```

Let me know once this is tested and committed, and we'll do **Task 4: Implement an idle hint indicator**!



after updating the python file

When replacing `find_matches` and `resolve_matches`, the `drop_and_refill` method was accidentally overwritten or indented outside the `Board` class.

PNG

Check if `drop_and_refill` is still in `game/board.py` by running this command in PowerShell:

PowerShell

```powershell
Select-String -Path .\game\board.py -Pattern "def drop_and_refill"
```

Fix: Restore `drop_and_refill`

Add `drop_and_refill` back directly inside the `Board` class in `game/board.py` (place it right after `resolve_matches` and before `is_animating` or `render`):

Python

```python
    def drop_and_refill(self):
        for c in range(GRID_SIZE):
            empty_slots = 0
            for r in range(GRID_SIZE - 1, -1, -1):
                if self.grid[r][c] is None:
                    empty_slots += 1
                elif empty_slots > 0:
                    gem = self.grid[r][c]
                    gem.target_row = r + empty_slots
                    gem.target_y = (r + empty_slots) * TILE_SIZE
                    self.grid[r + empty_slots][c] = gem
```

```
                    self.grid[r][c] = None

            for r in range(empty_slots):
                color = random.choice(GEM_COLORS)
                new_gem = Gem(color, r, c)
                new_gem.current_y = (r - empty_slots) * TILE_SIZE
                self.grid[r][c] = new_gem
```

Make sure the indentation matches the other methods in `class Board` (4 spaces before `def drop_and_refill(self):`).

Verification

Save `game/board.py` and run:

PowerShell

```
python main.py
```

The game should now launch without errors. Once you confirm it opens, we can verify Task 3 gameplay and move straight to Task 4.

lets head to task 4

Task 4: Implement an Idle Hint Indicator

To implement Task 4, we need to:

1. Detect a valid adjacent swap move on the board (`find_possible_move`).
2. Track the timestamp of the last player interaction.
3. If no action occurs for > 5 seconds, render a shimmering/pulsing highlight around the two swappable gems.
4. Reset the timer as soon as the player interacts with the board or while gems are animating.

1. Code Changes in `game/board.py`

Open `game/board.py` and apply the following additions:

A. In `Board.__init__` and `Board.reset`

Add `self.last_action_time`:

Python

```
        self.last_action_time = pygame.time.get_ticks()
```

Add this inside `__init__` and also inside `reset()`.

B. Add Helper Methods in `Board`

Add `reset_idle_timer` and `find_possible_move` inside the `Board` class:

Python

```
    def reset_idle_timer(self):
        self.last_action_time = pygame.time.get_ticks()

    def find_possible_move(self):
        for r in range(GRID_SIZE):
            for c in range(GRID_SIZE):
                for dr, dc in [(0, 1), (1, 0)]:
                    nr, nc = r + dr, c + dc
                    if nr < GRID_SIZE and nc < GRID_SIZE:
                        if self.grid[r][c] is None or self.grid[nr][nc] is None:
                            continue
                        # Temporarily swap gems to check for a match
                        self.grid[r][c], self.grid[nr][nc] = self.grid[nr][nc], self.
                        matches = self.find_matches()
                        self.grid[r][c], self.grid[nr][nc] = self.grid[nr][nc], self.

                        if matches:
                            return ((r, c), (nr, nc))
        return None
```

C. Update `Board.update`

Keep the timer fresh if gems are still falling or a gem is currently selected:

Python

```python
    def update(self):
        animating = False
        for r in range(GRID_SIZE):
            for c in range(GRID_SIZE):
                if self.grid[r][c]:
                    self.grid[r][c].update()
                    if self.grid[r][c].is_animating():
                        animating = True

        if animating or self.selected is not None or self.is_game_over():
            self.reset_idle_timer()
```

D. Update `Board.render`

At the end of `def render(self, surface):` (right after drawing the selected gem outline), add the pulsing idle hint:

Python

```python
        # Idle Hint Indicator (after 5 seconds of inactivity)
        now = pygame.time.get_ticks()
        if now - self.last_action_time >= 5000 and not self.is_animating() and not se
            hint_pair = self.find_possible_move()
            if hint_pair:
                pulse = int(160 + 95 * math.sin(now * 0.007))
                hint_color = (255, max(80, min(255, pulse)), 40)
                for hr, hc in hint_pair:
                    hx = self.offset_x + hc * TILE_SIZE
                    hy = self.offset_y + hr * TILE_SIZE
                    hint_rect = pygame.Rect(hx + 2, hy + 2, TILE_SIZE - 4, TILE_SIZE
                    pygame.draw.rect(surface, hint_color, hint_rect, width=3, border_
```

2. Code Changes in `game/game_engine.py`

Reset the idle timer whenever the player clicks:

In `handle_click(self, mouse_pos)` in `game/game_engine.py`, add `self.board.reset_idle_timer()` right at the start:

Python

```python
    def handle_click(self, mouse_pos):
        self.board.reset_idle_timer()

        if self.board.is_game_over() or self.board.is_animating():
            return
        # ... rest of handle_click unchanged ...
```

3. Test the Idle Hint

Run:

PowerShell

```
python main.py
```

- Leave your mouse untouched for 5 seconds.
- A golden shimmering border will outline two adjacent gems that can be swapped to make a match.
- Click any tile to verify the hint disappears immediately and resets the 5-second countdown.

4. Commit Task 4

In PowerShell, commit this final task:

PDF

PowerShell

```
git add game/board.py game/game_engine.py
```

```
git commit -m "Task 4: Implement idle hint indicator after 5 seconds of inactivity"
```

Final Wrap-Up Checklist

1. **Record the "After" Video:**
    1. Record a 10 to 15-second gameplay video showing:
    PDF
        1. An invalid swap (moves count does not drop).
        2. A valid match and combo chain (scores multiplying).
        3. A 4-in-a-row glowing gem detonating a line.
        4. The idle hint appearing after 5 seconds.
2. **Push to Your Personal Fork:**

    PowerShell

    ```
    git push origin main
    ```

    (Ensure you do not submit a PR to `SETAPESU26` *).*
    PDF
3. **Save Deliverables to** `Lab-4` **folder** in your coursework lab repository:
    PDF
    1. `before` and `after` video files.
    PDF
    2. Updated code.
    PDF
    3. This chat transcript exported as a PDF or DOC.
    PDF